```cpp
//1. Horner's Rule

int main() {
    int n;
    cout << "Degree of polynomial: ";
    cin >> n;

    if (n < 0 || n > 99) {
        cout << "Invalid degree!\n";
        return 0;
    }

    double a[100];
    cout << "Coefficients (highest to lowest):\n";
    for (int i = 0; i <= n; i++) cin >> a[i];
    double x;
    cout << "Value of x: ";
    cin >> x;

    double result = a[0];
    for (int i = 1; i <= n; i++)
        result = result * x + a[i];
    cout << "f(" << x << ") = " << result << endl;
}

// 1. Horner's Rule
// Example: f(x) = x³ – 4x² + x + 6 at x = 2 (PDF এর হুবহু উদাহরণ)
// Input:
// 3
// 1 -4 1 6
// 2
// Output:
// f(2) = 0
```

```cpp
//2. Banker's Rounding
int main() {
    long double x; int n;
    cout << "Enter number: ";
    cin >> x;
    cout << "Enter significant digits: ";
    cin >> n;
    if (n < 1) {
        cout << "Invalid digits!\n";
        return 0;
    }
    if (x == 0) {
        cout << "Rounded value: 0\n";
        return 0;
    }
    fesetround(FE_TONEAREST); banker's rounding(round to nearest even)
    int p = floor(log10(fabsl(x)));          // exponent
    long double factor = pow(10.0L, n - 1 - p);
    long double ans = nearbyintl(x * factor) / factor;
    cout << "Rounded value: " << setprecision(n) << ans << endl;
}
```

- `fabsl(x)` ব্যবহারের কারণে negative number-এর magnitude-ও handle হয়।
- `FE_TONEAREST` এবং `nearbyintl()` halfway case-এ nearest even integer বেছে নেয়। যেমন, এক significant digit-এ `2.5 → 2`, `3.5 → 4`।

```cpp
// 2. Banker's Rounding
// Input:
// 3.14159
// 4
// Output:
// Rounded value: 3.142
// (PDF উদাহরণ: 3.14159 → 3.142 )
// আরেকটা (tie case):
// Input:
// 2.5
// 1
// Output:
// Rounded value: 2
// (2.5 → 2, কারণ 2 জোড়ে Banker's rule)
```

```cpp
//3. Bisection Method

double f(double x) {
    return x * x * x - x - 1;
}

int main() {
    double a, b, e;
    cout << "Enter a, b: ";
    cin >> a >> b;
    cout << "Enter tolerance: ";
    cin >> e;
    if (e <= 0) {
        cout << "Invalid tolerance!\n";
        return 0;
    }

    if (a > b) {
        double t = a;
        a = b;
        b = t;
    }

    double fa = f(a), fb = f(b);
    cout << setprecision(12);

    if (fa == 0) {
        cout << "Root = " << a << endl;
        return 0;
    }

    if (fb == 0) {
        cout << "Root = " << b << endl;
        return 0;
    }

    if ((fa > 0 && fb > 0) || (fa < 0 && fb < 0)) {
        cout << "No root is bracketed in this interval.\n";
        return 0;
    }
```

```
    int i = 0;
    while (b - a > e && i < 1000) {
        double mid = (a + b) / 2;
        double fm = f(mid);
        i++;
        if (fm == 0) {
            a = b = mid;
            break;
        }
        if ((fa > 0 && fm > 0) || (fa < 0 && fm < 0)) {
            a = mid;
            fa = fm;
        } else {
            b = mid;
        }
    }
    cout << "Approximate root = " << (a + b) / 2 << endl;
    if (b - a > e)
        cout << "Tolerance not reached within 1000 iterations.\n";
}
// 3. Bisection Method
// Example: x³ - x - 1 = 0, root between 1 and 2
// Input:
// Enter a, b: 1 2
// Enter tolerance: 0.001
// Output:
// Approximate root = 1.32470703125
```

```cpp
// 3. Bisection Method easy
double f(double x) {
    return x*x*x - x - 1;
}
int main() {
    double a, b, e;
    cout << "Enter a, b: ";
    cin >> a >> b;
    cout << "Enter tolerance: ";
    cin >> e;
    if (e <= 0) {
        cout << "Invalid tolerance!\n";
        return 0;
    }
    if (a > b) {
        double t = a;
        a = b;
        b = t;
    }
    if (f(a) == 0) {
        cout << "Root = " << a << endl; return 0;
    }
    if (f(b) == 0) {
        cout << "Root = " << b << endl; return 0;
    }

    if (f(a) * f(b) > 0) {
        cout << "No root is bracketed in this interval.\n"; return 0;
    }

    int i = 0;
    while ((b - a) > e && i < 1000) {
        double mid = (a + b) / 2;
        if (f(mid) == 0) {
            a = b = mid;
            break;
        }
        if (f(a) * f(mid) < 0)
            b = mid;
        else
            a = mid;
        i++;
    }
    cout << "Approximate root = " << (a + b) / 2 << endl;
    if ((b - a) > e)
        cout << "Tolerance not reached within 1000 iterations.\n";
}
```

```cpp
// 4. Newton-Raphson Method
double f(double x) {
    return x * x * x - x - 1;
}
double df(double x) {
    return 3 * x * x - 1;
}
int main() {
    double x0, e;
    cout << "Initial guess: ";
    cin >> x0;
    cout << "Tolerance: ";
    cin >> e;
    if (e <= 0) {
        cout << "Invalid tolerance!\n";
        return 0;
    }
    for (int i = 0; i < 1000; i++) {
        if (fabs(f(x0)) < e) {
            cout << "Root = " << x0 << endl; return 0;
        }
        double derivative = df(x0);
        if (fabs(derivative) < 1e-12) {
            cout << "Derivative is zero. Method stopped.\n"; return 0;
        }
        double x1 = x0 - f(x0) / derivative;
        if (!isfinite
(x1)) {
            cout << "Method failed to converge.\n"; return 0;
        }
        if (fabs(x1 - x0) < e || fabs(f(x1)) < e) {
            cout << "Root = " << x1 << endl; return 0;
        }
        x0 = x1;
    }
    cout << "Approximate root = " << x0 << endl;
    cout << "Maximum iterations reached.\n";
}
// 4. Newton-Raphson Method
// Example: x³ - x - 1 = 0, initial guess 1.5
// Input:
// 1.5
// 0.0001
// Output:
// Root = 1.32472
```

```cpp
//5. Newton Forward Interpolation

int main() {
    int n;
    cout << "Number of points: ";
    cin >> n;
    if (n < 2 || n > 50) {
        cout << "Invalid number of points!\n";
        return 0;
    }
    double x[50], y[50], d[50][50];
    cout << "Enter x and y:\n";
    for (int i = 0; i < n; i++)
        cin >> x[i] >> y[i];
    double h = x[1] - x[0];
    if (fabs(h) < 1e-12) {
        cout << "Invalid x values!\n";
        return 0;
    }
    for (int i = 2; i < n; i++) {
        if (fabs((x[i] - x[i - 1]) - h) > 1e-8) {
            cout << "x values must be equally spaced!\n";
            return 0;
        }
    }

    // Forward difference table
    for (int i = 0; i < n; i++)
        d[i][0] = y[i];
    for (int j = 1; j < n; j++)
        for (int i = 0; i < n - j; i++)
            d[i][j] = d[i + 1][j - 1] - d[i][j - 1];

    double xv;
    cout << "Interpolate at x = ";
    cin >> xv;
    double u = (xv - x[0]) / h;
    double sum = y[0], term = 1;
    for (int i = 1; i < n; i++) {
        term = term * (u - (i - 1)) / i;
        sum = sum + term * d[0][i];
    }
    cout << "f(" << xv << ") = " << sum << endl;
}
```

```
// 5. Newton Forward Interpolation

// Example: নিচের টেবিল থেকে x = 2.5 এ interpolate করো
// x    y
// 1    1
// 2    8
// 3    27
// 4    64

// Input:
// 4
// 1 1
// 2 8
// 3 27
// 4 64
// 2.5

// Output:
// f(2.5) = 15.625

// (এটা x³ polynomial, তাই 2.5³ = 15.625 — exact match)
```

```cpp
// 6. Newton Divided Difference
int main() {
    int n;
    cout << "Number of points: ";
    cin >> n;
    if (n < 2 || n > 50) {
        cout << "Invalid number of points!\n";
        return 0;
    }
    double x[50], y[50], d[50][50];
    cout << "Enter x and y:\n";
    for (int i = 0; i < n; i++)
        cin >> x[i] >> y[i];
    // Check for duplicate x values
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (fabs(x[i] - x[j]) < 1e-12) {
                cout << "All x values must be distinct!\n";
                return 0;
            }
        }
    }
    // Divided difference table
    for (int i = 0; i < n; i++)
        d[i][0] = y[i];
    for (int j = 1; j < n; j++) {
        for (int i = 0; i < n - j; i++) {
            d[i][j] =
                (d[i + 1][j - 1] - d[i][j - 1])
                / (x[i + j] - x[i]);
        }
    }
    double xv;
    cout << "Interpolate at x = ";
    cin >> xv;
    double sum = d[0][0], term = 1;
    for (int i = 1; i < n; i++) {
```

```
        term = term * (xv - x[i - 1]);
        sum = sum + term * d[0][i];
    }
    cout << "f(" << xv << ") = " << sum << endl;
}
// 6. Newton Divided Difference
// একই টেবিল, x = 2.5:
// Input:
// 4
// 1 1
// 2 8
// 3 27
// 4 64
// 2.5
// Output:
// f(2.5) = 15.625
// আরেকটা non-polynomial উদাহরণ (PDF-এর style):
// Input:
// 3
// 1 1
// 2 4
// 4 16
// 3
// Output:
// f(3) = 9
// (y = x² ধরলে 3² = 9 )
```